

Live Region Mutation Testing for Commercial Cyber-Physical System Development Tool Chain

Lehuan Zhang^a, Shikai Guo^{b,*}, Zixuan Wang^b, Xiaoyu Wang^b, Xiaochen Li^a, He Jiang^a

^aSchool of Software, Dalian University of Technology, Dalian, China

^bSchool of Information Science and Technology, Dalian Maritime University, Dalian, China

{lehuanzhang}@mail.dlut.edu.cn, {shikai.guo, 1120211459}@mail.dlut.edu.cn, {xiaochen.li,jianghe}@mail.dlut.edu.cn

Abstract—MathWorks Simulink, a commercial CPS development tool chain, is widely used as an industry standard for designing and analyzing system behavior and generating embedded code for deployment. However, bugs in Simulink can cause unexpected behaviors during model compilation, making their elimination critical. Existing methods face two key challenges: generating equivalent models with varied data flows (data flow equivalence) and creating diverse block types to comprehensively test the compiler (mutation diversity). To address these, we propose LION, a differential testing approach. LION ensures data flow equivalence by inserting “store-revert” block pairs between existing blocks and tackles mutation diversity by employing Markov Chain Monte Carlo (MCMC) sampling to generate diverse new blocks. Differential testing is then used to identify bugs. Experiments show LION outperforms state-of-the-art approaches like SLforge, SLEMI, and COMBAT, detecting 6-10 additional compiler bugs in two weeks. Over two months, LION uncovered and reported 16 valid bugs in the widely used stable version of Simulink.

Index Terms—Cyber-Physical System, Simulink, Equivalence Modulo Input, Differential Testing, Compiler Testing.

I. INTRODUCTION

As a de facto industrial standard [1], the commercial CPS development tool chain MathWorks Simulink has been widely used to design and analyze Simulink models (i.e., *block-diagrams* with connections) for simulating the behavior of the Simulink, in many safety-critical applications such as automotive [2] and aerospace [3]. Developers employ Simulink models to model the behavior of the target system. Each block represents a predefined function to manipulate the data passed through the Simulink model. When the simulation fits the expectation of developers, Simulink automatically generates embedded code that gets deployed in target applications. However, Simulink is a complex software system that may contain bugs [4, 5, 6]. These bugs cause compilation errors of Simulink models or inject subtle unexpected behaviors into the applications that use the embedded code [7].

The Simulink compiler is the core component enabling developers to share simulations as standalone executables. To identify flaws and improve stability, research primarily uses generation-based and mutation-based methods for testing. Generation-based approaches, like SLforge [5], randomly generate valid Simulink models, expecting identical results across different Simulink settings. If results vary, a bug may be present. Mutation-based methods, such as SLEMI [1] and COMBAT [8], systematically modify seed models while maintaining semantics, using Equivalent Model Input (EMI) testing. If outputs differ from the seed model, a compilation error is likely. EMI, a form of differential testing, has been shown to identify semantic and logic bugs that generation-based approaches may miss [1].

However, existing EMI approaches for Simulink compiler testing face two main challenges: *the data flow equivalence challenge* and

the mutation diversity challenge. The first challenge arises because these methods often mutate blocks in “zombie regions”—areas of a Simulink model that are not executed or processed under a given input. Mutations in zombie regions may be ignored by the Simulink compiler, as they do not influence data flow. To identify more bugs, it is critical to mutate blocks in “live regions”—areas that can affect the data flow and be processed by the compiler, thus increasing the likelihood of triggering bugs. The second challenge is that most mutation strategies focus on modifying or deleting existing blocks, rarely introducing new block types. Since the Simulink compiler uses different simulation modes for optimizing various block types, generating equivalent model variants with new blocks is a challenge.

While live region mutation has been studied for testing traditional programming language compilers (e.g., LLVM [9], JVM [10]), these methods cannot be directly applied to Simulink compiler testing. Triggering more Simulink compiler bugs by altering active regions remains challenging due to several unique features of the Simulink language. Unlike traditional languages with formal specifications (e.g., Pascal, C [5]), Simulink lacks comprehensive language specifications. Additionally, Simulink’s handling of “executed” (zombie regions) and “non-executed” (active regions) code differs from traditional languages; for instance, an *If* block outputs a default value even for unexecuted branches. Simulink also introduces complexities like explicit sampling times, where outputs vary at different intervals, and automatic block data type inference, where minor model changes can alter data types [1]. Existing Simulink compiler testing methods [1, 8] largely focus on zombie region mutations, with limited evidence on the effectiveness of live region mutation.

To test Simulink compiler effectively, we propose LION, a **LI**ve region **blo**ck mutation **ON** approach, which considers the features of Simulink languages and addresses the aforementioned two challenges for Simulink compiler testing. LION consists of three components: equivalent data flow generation, model mutation, and differential testing. Given a seed Simulink model, the equivalent data flow generation component inserts a type of specific block pairs named “store-revert” at arbitrary points between two existing blocks. Each *store* block records the data passed through it. These data are reverted at the exit of the *revert* block to maintain the value equivalence. Since such modifications could change the automatically inferred block datatypes and break the equivalence modulo input property, we use *Datatype Conversion* blocks to maintain the datatype equivalence. By inserting *store-revert* block pairs, LION generates equivalent data flows with the seed Simulink model thus addressing the data flow equivalence challenge. The model mutation component learns the relationship of blocks from a repository of valid Simulink models. It uses MCMC sampling strategy to generate a sequence of new blocks between each *store-revert* block pair based on the learned relationship. Comparing to modifying/deleting existing blocks or reusing

* Corresponding author (Shikai Guo)

existing block sequences, LION generates new block sequences by sampling from a large number of valid and diverse Simulink models without knowing the language specifications, thus addressing the mutation diversity challenge. Finally, LION runs the seed Simulink model and its variants with the same Simulink settings to find bugs caused by output inconsistency on all sample time points.

We evaluate LION on multiple Simulink versions, including an old version Simulink R2018a and the stable widely-used version Simulink R2021b. LION finds 11 bugs on the two Simulink versions in 200 CPU hours, which significantly outperforms state-of-the-art generation-based and mutation-based approaches in Simulink compiler testing. LION can find 6-10 more bugs than the baselines on two versions of Simulink in two weeks. LION can continuously find new bugs in the Simulink, that has been fully tested before release. To run LION for two months, we have reported 16 confirmed bugs for a stable widely-used version Simulink R2021b, of which 12 bugs are new bugs. A further analysis shows that both equivalent data flow generation component and model mutation component contribute to the effectiveness of LION, which help LION efficiently generate a large number of Simulink model variants with a mutation success rate around 0.45. These results indicate LION is effective to improve the reliability of systems, that use Simulink to design embedded systems.

In summary, the main contributions of this work are:

- We propose LION, which uses equivalent data flow generation and model mutation to tackle the challenges of Simulink compiler testing.
- We conduct extensive experiments, which shows that the bug-finding capability of LION significantly outperforms state-of-the-art approaches.
- We release LION as an open-source tool for Simulink compiler testing [11].

II. RELATED WORK

Recent research on Simulink has mainly focused on simulation acceleration [12], test coverage enhancement [13], and code generation optimization [14], while studies on CPS toolchain and Simulink compiler testing remain limited. Bugs in the Simulink compiler are often misidentified as model issues, leading to incorrect diagnoses and disrupting the development process. Therefore, improving test coverage and accuracy in Simulink compiler testing is essential for ensuring model correctness and boosting development efficiency. For CPS tool chain testing, prior works have primarily focused on using Simulink models for testing compilers via differential and EMI techniques. Tools such as CyFuzz [4] and SLforge [5] generate Simulink models to find faults, with SLforge being state-of-the-art in bug-finding capability. Mutation-based approaches like SLEMI and COMBAT [8] modify existing models to test compilers, though their bug-finding ability is constrained by their focus on specific regions. In contrast, our proposed approach, LION, employs MCMC sampling to generate diverse and complex model variants, targeting both live and inactive regions. This enables deeper testing of Simulink compilers and addresses limitations of prior methods such as COMBAT, which focuses solely on zombie regions. Experimental results demonstrate the superior performance of LION in detecting compiler errors.

In compiler testing for traditional programming languages (e.g., C, C++, Java), differential testing remains a cornerstone. Techniques like Randomized Differential Testing (RDT), Different Optimization Levels (DOL), and EMI approaches have been extensively used. Tools like Csmith [15] generates random programs to uncover compiler bugs, while other approaches mutate programs in dead and live regions to maintain equivalence modulo inputs. Additionally, recent

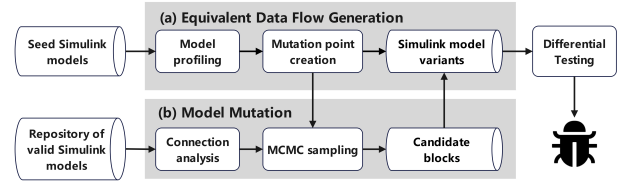


Fig. 1: The framework of LION.

works explore optimization sequence bugs and compiler-warning bugs. Unlike these efforts, which focus on traditional languages, our study emphasizes Simulink compiler testing and introduces LION as a novel live-region mutation strategy, outperforming existing Simulink-specific approaches in bug detection.

III. FRAMEWORK OF LION

In this section, we propose LION, a Live region block mutation approach, to address the two challenges for Simulink compiler testing. The framework of LION is illustrated in Fig. 1.

LION consists of three components: equivalent data flow generation, model mutation, and differential testing. The equivalent data flow generation component profiles the seed Simulink model and inserts *store-revert* block pairs at arbitrary points between two blocks. The *store* block records the data passed through it, while the *revert* block reverts the recorded data at its output ports to address the data flow equivalence challenge. Between each block pair, the model mutation component uses the MCMC sampling strategy to generate a sequence of new blocks by sampling existing blocks and connections from a repository of valid and diverse Simulink models, which addresses the mutation diversity challenge. Finally, differential testing is used to find Simulink compiler bugs.

A. Equivalent Data Flow Generation

Within this component, LION creates mutation points in a seed Simulink model. LION aims to alter the data flow of the seed Simulink model at each mutation point, while keeping the data flows equivalent. The equivalent data flow generation component has two steps, including model profiling and mutation point creation. We explain this phase with the Simulink model example in Fig. 2. Blocks in white are the original blocks in the seed Simulink model.

Model Profiling: This step executes the seed Simulink model and profiles the model by analyzing variables, their values, and their datatypes passed through input ports and output ports of each block. During the execution of a Simulink model, the Simulink compiler can identify both zombie regions and active regions. LION uses the Signal Range Coverage Tool [16] in Simulink for model profiling. For example, in Fig. 2, LION can capture the output of the *Abs* block (which is fed into the *Add* block) at each sample time point during the Simulink model execution.

Mutation Point Creation: After model profiling, LION randomly selects k points in the seed Simulink model as mutation points ($k = 10$ in this study). For example, in Fig. 2 LION selects a mutation point between *Abs* and *Add* blocks. LION can select mutation points in both live regions and zombie regions. However, in this study, we only add mutation points in live regions, because mutations in zombie regions might be eliminated if they do not affect the data flow of a Simulink model [17].

At each mutation point, LION inserts a pair of *store-revert* blocks to alter the data flow of the seed Simulink model. Specifically, LION adds two custom blocks at the mutation point, i.e., the *MATLAB Function Store* block and the *MATLAB Function Revert* block. The *MATLAB Function Store* block records variables and their values from

the output port of the *Abs* block. The *MATLAB Function Store* block has two output ports y and z . The port z connects with the input port v of the *MATLAB Function Revert* block, which is used to pass the recorded variables and values to the *Add* block through the output port w . Since all variables and their values from the *Abs* block have been transferred to the *Add* block, the data flow equivalence is kept.

However, Simulink model has the process of datatype inference, where a small modification may significantly change the automatically inferred datatypes of blocks and break the data flow equivalence, especially when live regions are modified. To maintain the equivalence modulo input property, we follow the existing study [1] to add a *Datatype Conversion* block before each block in the Simulink model. These blocks automatically specify the datatypes of blocks in Simulink models based on the datatypes parsed during profiling.

Between a *store-revert* block pair, LION creates mutation space for generating a sequence of blocks (e.g., the *Divide* block and the *First Order Hold* block in Fig. 2). These new blocks take variables and values recorded in the *MATLAB Function Store* block as input (delivered through the output port y), perform operations on the recorded variables, and pass results to the input port u of the *MATLAB Function Revert* block. These results are saved in the *MATLAB Function Revert* block locally and will not affect subsequent blocks.

Compared to existing Simulink compiler testing approaches that mainly mutate in zombie regions, LION significantly increases the space for Simulink model mutation by inserting *store-revert* block pairs at arbitrary points in the seed Simulink model. This strategy adds side effects on the seed Simulink model by changing its data flow. However, these side effects are reverted at the exit of the *MATLAB Function Revert* block to maintain the equivalence modulo input property. Since these *store-revert* block pairs are in live regions and alter the data flow, Simulink compiler has to optimize blocks in the new data flows. It helps more thoroughly test Simulink compiler.

B. Model Mutation

In this phase, LION automatically generates block sequences between each *store-revert* block pair using an MCMC sampling strategy for model mutation. MCMC approximates a target distribution by creating a Markov chain, where each state depends on the previous one, and the chain's long-run distribution mirrors the target. This enables statistical inference and is widely used to enhance mutation-based testing [18]. Unlike previous studies that reuse existing code snippets, we use MCMC to generate block sequences in Simulink, a block-diagram-based language with no formal specifications. The key idea is to build a Markov chain to capture the relationships between block connections in valid Simulink models, allowing LION to sample diverse and valid block sequences from a repository.

The model mutation process is outlined in Algorithm 1. LION takes a seed Simulink model M , a repository R_M of valid models, *store-revert* block pairs $\langle blockS, blockR \rangle$, and a parameter N specifying the number of blocks per pair. It outputs a mutated Simulink model M' . LION first analyzes block connections in M using the repository R_M (line 1). The repository provides a variety of candidate blocks for sampling, and since the connections between blocks are datatype-compatible, this enhances mutation success. We use SLforge to generate the repository of valid models. LION builds a block database *blockDB* from R_M and computes two probabilities: the block density distribution *densePr* and the transition probability *transPr*. *densePr* reflects the frequency of each block in R_M , while *transPr* represents the one-step transition probability from block b to block $\hat{b}$, calculated as the ratio of connections from b to $\hat{b}$ over the total connections from b .

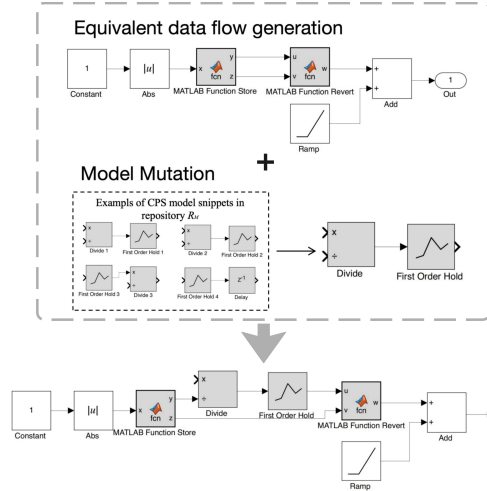


Fig. 2: An example of Simulink model variant generated by LION and the repository.

Algorithm 1 Model Mutation

Require: Seed Simulink model M
Valid Simulink model repository R_M
Store-revert block pairs $\langle blockS, blockR \rangle$
Number of blocks N to generate in each block pair

Ensure: Simulink model variant M'

- 1: $densePr, transPr, blockDB \leftarrow Parse(R_M)$
- 2: **for** $\langle blockS_i, blockR_i \rangle \in \langle blockS, blockR \rangle$ **do**
- 3: $headBlock \leftarrow SelectBlock(blockDB)$
- 4: $M' \leftarrow AddLine(M, blockS_i, headBlock)$
- 5: $curBlock = headBlock$
- 6: **for** $i = 0 \rightarrow N - 1$ **do**
- 7: $candiBlocks \leftarrow ConnectedBlocks(curBlock, blockDB)$
- 8: $accRates \leftarrow CalcAcc(densePr, transPr, candiBlocks)$
- 9: $nextBlock \leftarrow ChooseBlock(candiBlocks, accRates)$
- 10: $M' \leftarrow AddLine(M', curBlock, nextBlock)$
- 11: $curBlock = nextBlock$
- 12: **end**
- 13: $M' \leftarrow AddLine(M', curBlock, blockR_i)$
- 14: **end**
- 15: $M' \leftarrow DataTypeInference(M')$
- 16: **return** (M')

After analyzing R_M , LION starts to generate new blocks between each *store-revert* block pair. Initially, LION randomly selects a block in *blockDB* as the head block to increase the mutation diversity (line 3). The head block is connected with the *store* block (line 4). Taking the head block as the current block, LION collects all blocks in *blockDB* that the current block connects with (line 7). LION calculates an acceptance rate *accRate*, representing the probability to accept $\hat{b}$ as the next block when the current block is b (line 8).

$$accRate(b \rightarrow \hat{b}) = \min(1, \frac{\theta(\hat{b})q(b|\hat{b})}{\theta(b)q(\hat{b}|b)}), \quad (1)$$

where $\theta(b)$ and $\theta(\hat{b})$ are the probability density distribution of blocks b and $\hat{b}$, respectively, and $q(b|\hat{b})$ and $q(\hat{b}|b)$ represent their transition probability. The intuition of this formula is, blocks that are frequently used and connected with block b should have a higher chance to be accepted as the next block. The overall significance of the division is to assess the relative likelihood of accepting the candidate block $\hat{b}$, thereby determining whether to accept it

and ensuring that the Markov chain ultimately converges to the target distribution. LION randomly selects the next block taking into consideration of the acceptance rate of each candidate block (line 9), i.e., blocks with higher acceptance rates have a higher chance to be selected. In an extreme case, there could be no block to select (e.g., the current block is a *sink* block). LION will randomly select a block in *blockDB* as the next block. After selection, LION takes the selected block as the current block for the next round of sampling (line 11). The sampling in each *store-revert* block pair stops when the number of selected blocks reaches a certain number N . The output port of the last block in the sequence is connected with the *revert* block (line 13).

After generating a Simulink model variant, LION propagates datatype information from *store* blocks to the newly generated blocks along the connections using forward propagation (line 15). If the datatype is not supported by a block, we follow SLEMI to add a *Datatype Conversion* block before it [1]. The algorithm ends by returning a Simulink model variant of the seed Simulink model. In this stage, we generate Simulink model variants with different types of blocks to exercise the Simulink compiler thoroughly, as it may use various optimization strategies to optimize these variants.

After the mutation process, LION generates a large number of mutated versions of the seed Simulink models. Next, LION filters these mutants. When mutated Simulink models fail to pass Simulink compilation, LION discards such mutants and retains only those that successfully compile for subsequent differential testing.

C. Differential Testing

LION employs EMI (a form of differential testing) to evaluate the Simulink compiler. As illustrated in Fig. 3, given the original inputs of the seed model, equivalent variant models are expected to produce identical outputs. To this end, LION executes the seed model and its variants under the same Simulink configuration (e.g., *Normal* mode or *Accelerator* mode) and compares their outputs. We use the Simulation Data Inspector of Simulink to show this process graphically and intuitively.

Specifically, for a seed Simulink model M and each of its variants M' , let $t \in [t_0, t_n]$ denotes a sampling time point within the testing time interval. The output of model M at sampling time t_i under *Normal* mode is denoted as $O_M^N(t_i)$. The output of model M at sampling time t_i under *Accelerator* mode is denoted as $O_M^A(t_i)$. Similarly, the outputs of M' at sampling time t_i in *Normal* and *Accelerator* mode are denoted as $O_{M'}^N(t_i)$ and $O_{M'}^A(t_i)$. If there exists a sampling time t_i such that:

$$O_M^N(t_i) \neq O_{M'}^N(t_i) \quad \text{or} \quad O_M^A(t_i) \neq O_{M'}^A(t_i),$$

then the variant model M' is marked to have triggered a bug. The above process is repeated for all variant Simulink models generated from each seed Simulink model, ultimately identifying all variants marked as triggering bugs.

IV. EVALUATION

In this section, we evaluate our approach LION for Simulink compiler testing. First, we compare LION with state-of-the-art approaches to assess their overall effectiveness in finding Simulink compiler bugs. Second, we evaluate how the use of the equivalent data flow generation component and the model mutation component affect the effectiveness of LION. Last, we run LION on the stable widely-used Simulink version to analyze its bug-finding capability in practice. Specifically, we aim to answer the following research questions:

RQ1 *How is the bug-finding capability of LION compared against state-of-the-art approaches?*

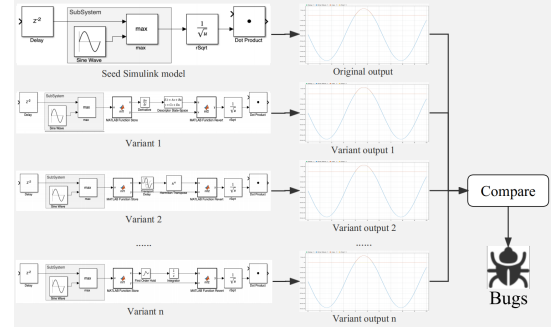


Fig. 3: Schematic diagram of differential testing.

RQ2 *What is the impact of using the equivalent data flow generation component and the model mutation component on the effectiveness of LION?*

RQ3 *Can LION steadily find new bugs in the stable widely-used Simulink version?*

A. Seed Simulink Models And Testing Strategy

LION uses seed Simulink models to generate model variants. Building on existing research, LION utilizes both real-world models and SLforge-generated models as seeds. Real-world models are sourced from a corpus of 1,000 publicly available CPS projects [19]. LION filters out models that could not run in our environment, retaining 357 Simulink models. To supplement this limited number, SLforge, a Simulink model generator, is employed to create additional seeds, following prior methods [1]. These models contain 1–5 subsystems with 30–100 blocks per subsystem, resulting in seed models with 100–3,000 blocks. SLforge’s generation time increases significantly with block count due to connection corrections, potentially dominating the testing process. Moreover, models exceeding this scale rarely trigger compiler bugs, as shown in preliminary evaluations. This range is chosen to balance efficiency and effectiveness in highlighting LION’s ability to generate bug-triggering variants.

In the testing strategy, LION generates one seed Simulink model at a time and feed it to LION to create model variants. These seed models serve as a repository for model mutation, with each generated seed added to the repository. The repository is continuously updated with new valid models. Once variants are generated, LION runs both the seed and its variants with the same Simulink settings. First, LION tests all models in *Normal* simulation mode to find bugs, then in *Accelerator* mode. To isolate equivalence modulo input’s impact from differential testing, LION does not compare the outputs between *Normal* and *Accelerator* simulation modes (i.e., the strategy of plain differential testing). When all Simulink models have been executed, LION generates the next seed Simulink model. Testing continues until the predefined period ends.

B. Evaluation Criterion

When a bug is detected, LION compares its failed assertion and back-trace with those of previously identified bugs by LION. If they match, LION classifies it as a duplicate. However, some miscompilation bugs do not have the information of failed assertion and back-trace. De-duplicating such bugs is still an open research question.

Initially, these bugs are considered unique and are provided to Simulink developers for further analysis. For each unique bug, LION reduces the corresponding Simulink model by iteratively removing blocks until it identify the minimal model that triggers the bug [18, 20]. Then, we create a bug report for each bug to

explain the symptom of the bug and submit it to the MathWorks’ bug reporting platform [6]. We mainly get two types of feedback from the MathWorks support team for a bug report, i.e., *new* and *known*, where “new” means this is a new bug in Simulink and “known” means the bug is duplicate with existing bugs in their bug repository. Since the complete list of bug reports is neither available nor traceable in the MathWorks’ bug reporting platform, we publish our reported Simulink models for reproducing our identified bugs [11].

C. Implementation And Settings

We implement LION as a Matlab program. Given a seed Simulink model, we configure LION to generate five Simulink model variants, which is the same setting as the existing study SLEMI [1] and COMBAT [8]. In addition, LION has two unique parameters, i.e., the number of mutation points (i.e., *store-revert* block pairs) to insert k (as shown in Section III-A) and the number of blocks to generate per mutation point N (as shown in Algorithm 1). During mutation, we configure LION to insert 10 mutation points in the seed Simulink model. At each mutation point, LION randomly generates 5 blocks. These parameters were selected based on experimental comparisons.

We performed our experiments with a computer running Windows 10 64-bit system with a 2.10 GHz Intel Core i9 processor and 120 GB of memory. The implementation of LION and experimental results are available at GitHub [11].

D. Comparison With Baselines (RQ1)

To assess the effectiveness of LION, we compare the bug-finding capability of LION with the state-of-the-art approaches:

- 1) SLforge [5] generates random Simulink models with controllable size and complexity, though it supports only a subset of Simulink specifications due to incomplete documentation.
- 2) SLEMI [1] applies equivalence modulo input (EMI) techniques to mutate seed Simulink models and generate variants for compiler testing.
- 3) COMBAT [8] extends SLEMI by modifying zombie regions and adding blocks to enhance variant generation, improving bug detection through EMI testing.

Since SLforge is a Simulink model generator that emphasizes plain differential testing. For each generated Simulink model by SLforge, we run it in both *Normal* and *Accelerator* simulation modes to identify the output inconsistency between the two modes. For SLEMI, COMBAT, and LION, we use SLforge-generated seed models and run each seed and its variants in a single simulation mode (*Normal* or *Accelerator*) to isolate the impact of equivalence modulo input.

We run each approach with an evaluation period of 200 CPU hours [1, 8], including the time to generate Simulink models, mutation (if necessary), and differential testing. We choose two Simulink versions for testing, i.e., Simulink R2018a and Simulink R2021b. The reason is, Simulink R2018a has been used in the related work [1, 8], while Simulink R2021b is the stable widely-used version during this study. We reproduce SLforge, SLEMI and COMBAT with the source code provided in their studies and use the default configurations in the source code or specified in the corresponding papers. We have cross-reviewed the implementation and resolve deployment issues.

Table I presents the bugs found by different approaches on the two Simulink versions. As shown in Table I, the baselines only find 6 or fewer bugs. Specifically, SLforge, SLEMI, and COMBAT found 1, 3, and 6, respectively. In contrast, LION finds 11 bugs on the two Simulink versions in the testing period, that is at least twice what baselines find. SLforge only find 1 bug in 200 CPU hours. SLforge detects a bug in Simulink R2018a where a model worked in normal

TABLE I: Bugs Found by LION and Baselines.

	Simulink R2018a	Proper Simulink version(Simulink R2021b)	Total
SLforge	1	0	1
SLEMI	2	1	3
COMBAT	4	2	6
LION	3	8	11

TABLE II: Impact of different components in LION.

	LION _{dis_dflow}		LION _{dis_mcmc}		LION	
	New	Known	New	Known	New	Known
Bugs	0	2	0	5	2	6

mode but failed in accelerated mode, a bug also identified by LION and SLEMI due to LION’s use of SLforge-generated seed models. While LION and COMBAT identified overlapping bugs, COMBAT’s effectiveness was limited to Simulink R2018a, with many bugs fixed in later versions. COMBAT’s reliance on zombie regions further hindered its performance in newer Simulink versions.

LION, on the other hand, was able to uncover additional potential bugs in Simulink by overcoming challenges related to data flow equivalence and mutation diversity. Notably, LION found more bugs in Simulink R2021b, as developers primarily fix bugs in newer versions. If a bug in Simulink R2018a does not impact relatively newer versions, it is rarely confirmed by the developers. In particular, LION finds more bugs by generating new mutation space in live region with different control/data flow characteristics.

E. Impact Of Different Components (RQ2)

LION has two important components: (1) the equivalent data flow generation component, which generates different but equivalent data flows for a seed Simulink model (Section III-A), and (2) the model mutation component, which uses the MCMC sampling strategy to generate diverse blocks (Section III-B). To answer RQ2, we assess the impact of these two components on the effectiveness of LION.

We create two variants of LION, denoted as LION_{dis_dflow} and LION_{dis_mcmc}. LION_{dis_dflow} disables the impact of the equivalent data flow generation component. Instead of creating *store-revert* block pairs, we use SLEMI and COMBAT to randomly create a new signal path terminated by a *sink* block in live regions (i.e., the fourth mutation strategy in SLEMI [1]). In the new path, blocks are generated by the MCMC sampling strategy. LION_{dis_dflow} investigates whether the data flow modification strategy in LION is more effective than the existing strategy. LION_{dis_mcmc} disables the impact of the model mutation component by replacing the MCMC sampling strategy with a random sampling strategy, which randomly selects blocks from a repository and adds them into *store-revert* block pairs. We run LION and the additional variants for 200 CPU hours, and measure their bug-finding capability. Since developers mainly work on the stable Simulink version, we test Simulink R2021b in the following RQs.

As shown in Table II, both components impact the effectiveness of LION. The equivalent data flow generation component helps LION thoroughly test Simulink compiler by altering the data flow that has to be optimized in live regions. The model mutation strategy generates Simulink model variants with different types of blocks, which allows Simulink compiler to use various optimization strategies to optimize them. When we disable the impact of a component, the bug-finding capability of LION drops. LION_{dis_dflow} and LION_{dis_mcmc} find 2 and 5 bugs on Simulink R2021b, respectively. LION_{dis_dflow} finds fewer bugs than LION_{dis_mcmc}. It means comparing with existing strategy in SLEMI, the *store-revert* block pairs inserted in the seed Simulink model significantly improve the effectiveness of LION.

F. Effectiveness In Finding New Bugs (RQ3)

LION is tested on Simulink R2021b, a stable and widely-used version, for two months. Developers prioritize fixing bugs in such versions, making the bugs discovered by LION more relevant and often new. Using the strategy in Section IV-A, all bugs identified by LION are submitted to MathWorks’ bug reporting platform for verification ((as shown in Section IV-B). Due to the time required for a full run, comparisons with SLforge, SLEMI, and COMBAT are not conducted.

We manually classify the submitted bugs from three dimensions:

- 1) The feedback from the MathWorks support team (Fb): Indicating whether the bug was new (N) or known (K).
- 2) The Discovery Point (Dp): The stage of detection, either during compilation (C) or runtime (R).
- 3) The Symptom Type (St): Categorized as compiler prompt blur (B), inconsistent compilation prompts (I), time vector errors (T), or data inconsistency errors (D), based on back-tracing and MathWorks’ feedback.

As shown in Table III, LION found 16 confirmed bugs in Simulink R2021b in two months, of which 12 bugs are new and 4 bugs are known. Each bug has a unique ID from MathWorks. Simulink models triggering these bugs are in our replication package.

LION can find bugs in both the compilation stage (C) and the runtime stage (R). During compilation, LION found 7 bugs, which are related to compiler prompt blur (B) and inconsistent compilation prompts (I). 9 bugs are found in the runtime stage. They are related to time vectors inconsistent error (T) and data inconsistent error (D). To better illustrate these issues, we present several representative bugs identified by LION. For simplicity, all Simulink models that trigger bugs have been reduced.

TSC 05359223: *Compilation error prompts an unreasonable exception.* This type of bugs means that warning or error messages prompted by Simulink are vague or misleading. As illustrated in Fig. 4, the seed Simulink model initially compiles successfully. However, when we generated Simulink model variants for compilation, Simulink showed an error message for datatype inference “data type mismatch: the internal rule determines the output of ‘model/Subsystem/ PID 2dof. . . is ‘ufix32_En22’; however, it is driving a signal of data type ‘double’”. This error message may not help trace the bug, since *PID 2dof* is a complex and encapsulated block in Simulink which can be hardly monitored and manipulated. The MathWorks support team has confirmed this bug and created a request for developers to enhance the error message. This bug is classified as compiler prompt blur, because the error message hinders developers for analyzing the bug.

TSC 05416819: *Time Vectors Inconsistent Error.* This type of bugs reflects errors in the sample time inference diagnostic. A typical example is shown in Fig. 5. The cause of this bug lies in Simulink incorrectly passing data from the *Discrete FIR Filter* block to the *Data Type Conversion* block, leading to incorrect time inference in the latter block. This bug has been confirmed. SLforge, SLEMI, and COMBAT may not generate such Simulink models, as it requires complex mutations involving long sequences of blocks.

TSC 05382868: *Data Inconsistent Error.* This type of bugs means that output data of two equivalent Simulink models are different. They are serious bugs since such bugs could inject unexpected behaviors in target Simulink applications. LION finds 8 bugs of this type. An example of Simulink model triggers this type of bug is presented in Fig. 6. Simulink compiler cannot correctly optimize Simulink models in the *Accelerator* simulation mode if a Simulink model has the *max*

TABLE III: Bugs found by LION in an evaluation period of two months.

#	TSC	Summary	Fb	Dp	St
1	5318668	S-Function too many output arguments	N	C	B
2	5359149	Equivalent mathematical function modules compile different prompts	N	C	I
3	5359223	Compilation error prompts an unreasonable exception	N	C	B
4	5370105	Exception in accelerate mode for if block	N	C	I
5	5382868	Data results different exception of max block	N	R	D
6	5394704	Min block produce incorrect results in accelerator modes with NaN input	K	R	D
7	5403026	Zero-crossing signal exception of the min block	N	C	I
8	5407559	Unknown module acceleration mode is abnormal	N	C	I
9	5411035	The Observer Block acceleration mode is not detected	N	C	I
10	5416819	Time vector inconsistency error of DTC block	N	R	T
11	5422544	Inconsistent Delay module data in different modes	N	R	D
12	5422545	Error in Sum module with onconsistent data result	N	R	D
13	5434051	Sinwave module data is inconsistent in different compilation	K	R	D
14	5436071	Ramp block is affected and produces different data in different modes	N	R	D
15	5445811	Problem of inconsistent data in the RT module	K	R	D
16	5445813	Data inconsistency errors when running the Product module	K	R	D

TSC = Technical Support Case number from MathWorks; Fb = feedback from MathWorks (N = new bug, K = known bug); Dp = discovery point (C = compilation stage, R = runtime stage); St = symptom of bugs (B = compiler prompt blur, I = inconsistent compilation prompts, T = time vectors inconsistent, D = data inconsistent error,).

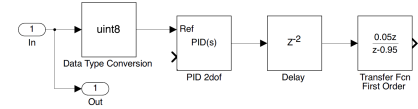


Fig. 4: TSC 05359223: Compilation error prompts an unreasonable exception.

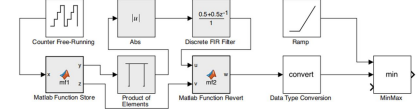


Fig. 5: TSC 05416819: Time Vectors Inconsistent Error.

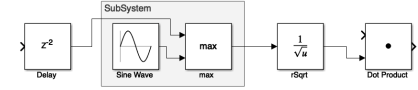


Fig. 6: TSC 05382868: Data inconsistent error.

block with multiple input ports and *NaN* is the first value passed into it. The MathWorks support team considers this bug as serious, since it affects Simulink versions from R2015a to R2021b. The bug was fixed immediately after our submission. However, it was not found in existing studies.

V. CONCLUSIONS AND FUTURE WORK

In this study, we propose LION, an approach for automatically generating equivalent Simulink models to test Simulink compilers. LION includes components for generating equivalent data flows and diverse model variants, addressing the challenges in compiler testing. LION modifies the data flow of a seed Simulink model while preserving equivalence, then uses an MCMC sampling strategy to generate diverse blocks. These models are used for equivalence modulo input testing. Evaluation results show that LION outperforms state-of-the-art methods in bug detection, finding 16 confirmed bugs in Simulink R2021b, 12 of which were previously unknown. Future work includes testing LION with more CPS development toolchains and analyzing the impact of testing time on bug-finding effectiveness.

ACKNOWLEDGMENT

This work was supported by the key research and development project of Liaoning Province (2024JH2/102400059), the National Natural Science Foundation of China (No.62472062, No.62202079), the Dalian Science and technology Innovation Fund project (No.2024JJ12GX022).

REFERENCES

- [1] S. A. Chowdhury, S. L. Shrestha, T. T. Johnson, and C. Csallner, "Slemi: Equivalence modulo input (emi) based mutation of cps models for finding compiler bugs in simulink," in *Int'l Conf. on Software Eng. (ICSE)*. Washington, USA: IEEE, 2020, pp. 335–346.
- [2] MathWorks Inc, "Products and services," <http://www.mathworks.com/products/>, 2018.
- [3] J. Zander, I. Schieferdecker, and P. J. Mosterman, *Model-Based Testing for Embedded Systems*. Boca Raton, USA: CRC press, 2017.
- [4] S. A. Chowdhury, T. T. Johnson, and C. Csallner, "Cyfuzz: A differential testing framework for cyber-physical systems development environments," in *Cyber Physical Systems Design, Modeling, and Evaluation*. Cham: Springer International Publishing, 2017, pp. 46–60.
- [5] S. A. Chowdhury, S. Mohian, S. Mehra, S. Gawsane, T. T. Johnson, and C. Csallner, "Automatically finding bugs in a commercial cyber-physical system development tool chain with slforge," in *Int'l Conf. on Software Eng. (ICSE)*. New York, NY, USA: ACM, 2018, pp. 981–992.
- [6] MathWorks Inc, "Bug report platform," <https://www.mathworks.com/support/bugreports/>, 2018, (last access: 16/03/2022).
- [7] A. Brillout, N. He, M. Mazzucchi, D. Kroening, M. Purandare, P. Rümmer, and G. Weissenbacher, "Mutation-based test case generation for simulink models," *Int'l Symposium on Formal Methods for Components and Objects (FMCO)*, vol. 6286, p. 208–227, 2009.
- [8] S. Guo, H. Jiang, Z. Xu, X. Li, Z. Ren, Z. Zhou, and R. Chen, "Detecting simulink compiler bugs via controllable zombie blocks mutation," in *Joint European Software Eng. Conf. and Symposium on the Foundations of Software Eng. (ESEC/FSE)*, 2022, pp. 1061–1072.
- [9] C. Sun, V. Le, and Z. Su, "Finding compiler bugs via live code mutation," 2016, p. 849–863. [Online]. Available: <https://doi.org/10.1145/2983990.2984038>
- [10] Y. Chen, T. Su, and Z. Su, "Deep differential testing of jvm implementations," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1257–1268.
- [11] LION, "Lion replication package," <https://github.com/Z-Lighthouse/LION>, 2024.
- [12] Y. Cheng, Z. Yu, Z. Su, T. Chen, X. Zhang, and Y. Jiang, "Accmos: Accelerating model simulation for simulink via code generation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [13] Z. Su, Z. Yu, D. Wang, R. Wang, Y. Tao, and Y. Jiang, "Cftcg: Test case generation for simulink model through code based fuzzing," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [14] Z. Yu, Z. Su, Y. Jiang, A. Cui, and R. Wang, "Efficient code generation for data-intensive simulink models via redundancy elimination," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [15] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*. New York, NY, USA: ACM, 2011, pp. 283–294.
- [16] MathWorks Inc, "Signal range coverage tool," <https://ww2.mathworks.cn/help/slcoverage/ug/types-of-model-coverage.html>, 2022, (last access: 16/03/2022).
- [17] C. Sun, V. Le, and Z. Su, "Finding compiler bugs via live code mutation," in *Int'l Conf. on Object-Oriented Programming, Systems, Languages, and Applications (SIGPLAN)*. New York, NY, USA: ACM, 2016, pp. 849–863.
- [18] Y. Tang, H. Jiang, Z. Zhou, X. Li, Z. Ren, and W. Kong, "Detecting compiler warning defects via diversity-guided program mutation," *IEEE Trans. on Software Eng. (TSE)*, pp. 1–1, 2021.
- [19] S. A. Chowdhury, L. S. Varghese, S. Mohian, T. T. Johnson, and C. Csallner, "A curated corpus of simulink models for model-based empirical studies," in *Proceedings of the 4th International Workshop on Software Engineering for Smart Cyber-Physical Systems*, 2018, pp. 45–48.
- [20] H. Jiang, Z. Zhou, Z. Ren, J. Zhang, and X. Li, "Ctos: Compiler testing for optimization sequences of llvm," *IEEE Trans. on Software Eng. (TSE)*, pp. 1–1, 2021.